

Comparação de Algoritmos Gulosos e Programação Dinâmica na Resolução de Problemas Clássicos

Objetivo:

Os alunos deverão implementar e comparar um algoritmo guloso e um algoritmo de programação dinâmica para resolver o mesmo problema clássico, analisando a eficiência e eficácia de cada abordagem.

Problemas Clássicos Sugeridos:

1. **Problema da Mochila (Knapsack Problem)**
2. **Problema do Caminho Mínimo (Shortest Path Problem)**
3. **Problema da Seleção de Atividades (Activity Selection Problem)**
4. **Problema de Partição (Partition Problem)**
5. **Problema do Troco (Coin Change Problem)**
6. **Problema da Subsequência Comum Mais Longa (Longest Common Subsequence - LCS)**

Estrutura do Relatório:

1. **Introdução:**
 - Explicação dos conceitos de algoritmos gulosos e programação dinâmica.
 - Descrição do problema escolhido e sua relevância.
2. **Implementação:**
 - Detalhar a implementação do algoritmo guloso para o problema escolhido.
 - Detalhar a implementação do algoritmo de programação dinâmica para o mesmo problema.
 - Incluir código e/ou pseudo-código para explicação.
3. **Análise Teórica:**
 - Analisar a complexidade temporal e espacial de cada abordagem.
 - Discutir os casos em que cada abordagem é mais eficiente ou adequada.
4. **Experimentos e Resultados:**
 - Testar ambos os algoritmos com diferentes conjuntos de dados.
 - Comparar o desempenho (tempo de execução, uso de memória) entre o algoritmo guloso e o de programação dinâmica.
 - Apresentar os resultados em gráficos e tabelas.
5. **Discussão:**
 - Comparar os resultados obtidos e discutir as diferenças observadas.
 - Discutir as vantagens e desvantagens de cada abordagem.
 - Propor situações práticas onde uma abordagem seria preferível à outra.
6. **Conclusão:**
 - Resumir as principais descobertas do trabalho.

- Refletir sobre o aprendizado obtido com a implementação e comparação dos algoritmos.
7. **Referências:**
- Listar todas as fontes e referências bibliográficas utilizadas.

Metodologia:

1. **Formação de Grupos.**
2. **Escolha do Problema:** Cada grupo escolhe, preferencialmente, um dos problemas clássicos listados. Entretanto, o grupo tem autonomia para escolher outro problema, desde que não tenha sido previamente escolhido por outro grupo.
3. **Desenvolvimento:** Os alunos desenvolvem e testam as implementações dos algoritmos.
4. **Documentação:** Produção do relatório final conforme a estrutura previamente sugerida.
5. **Apresentação:** Cada grupo apresenta seus resultados e análises para a turma.

Avaliação:

- **Relatório:**
 - **Corretude das Implementações:** 20%
 - **Fundamentação e Análise Teórica:** 15%
 - **Experimentos, Resultados e Discussão:** 25%
 - **Estrutura, Formatação e Conclusão:** 10%
- **Apresentação:**
 - **Domínio Técnico e Clareza:** 10%
 - Capacidade da equipe de explicar a lógica por trás da implementação dos dois algoritmos de forma didática e clara, demonstrando que realmente compreenderam o que foi programado.
 - **Exposição dos Resultados e Comparações:** 10%
 - Tradução da análise teórica e os gráficos de execução em uma conclusão compreensível, justificando qual foi o melhor algoritmo para o problema escolhido.
 - **Dinâmica Prática com a Turma:** 10%
 - Elaboração e condução de um exemplo guiado, exercício rápido ou atividade interativa envolvendo os demais colegas para demonstrar o funcionamento do problema clássico e a aplicação dos algoritmos na prática.

O que deve ser entregue:

1. **Códigos fontes**
2. **Relatório em PDF**
3. **Apresentação**

Breve detalhamento dos problemas sugeridos:

1. **Problema da Mochila (Knapsack Problem):**
 - **Guloso:** Escolher itens com a melhor razão valor/peso até encher a mochila.
 - **Programação Dinâmica:** Utilizar uma tabela para calcular o valor máximo possível para cada capacidade.
2. **Problema do Caminho Mínimo (Shortest Path Problem):**
 - **Guloso:** Algoritmo de Dijkstra.
 - **Programação Dinâmica:** Algoritmo de Floyd-Warshall.
3. **Problema da Seleção de Atividades (Activity Selection Problem):**
 - **Guloso:** Selecionar a atividade que termina mais cedo e que não conflita com as atividades já selecionadas.
 - **Programação Dinâmica:** Utilizar uma tabela para considerar todas as combinações possíveis de atividades.
4. **Problema de Partição (Partition Problem):**
 - **Guloso:** Tentativas de dividir o conjunto de números com base em heurísticas, embora essa abordagem não garanta a solução ótima.
 - **Programação Dinâmica:** Utilizar uma tabela para verificar se o conjunto pode ser dividido em dois subconjuntos com soma igual.
5. **Problema do Troco (Coin Change Problem):**
 - **Guloso:** Sempre pegar a maior moeda possível.
 - **Programação Dinâmica:** Utilizar uma tabela para calcular o número mínimo de moedas necessário para cada valor.
6. **Problema da Subsequência Comum Mais Longa (Longest Common Subsequence - LCS):**
 - **Guloso:** Embora menos comum, pode-se tentar heurísticas, como o alinhamento das sequências.
 - **Programação Dinâmica:** Utilizar uma matriz para calcular a subsequência comum mais longa.